



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



FOOD FASTER - SMART RESTAURANT & DIGITAL SERVICE PLATFORM

AN INTERNSHIP REPORT

Submitted by

P LAKSHMI NARASIMHA – 20221CDV0009

Under the Guidance of,

Mr. Syed Mohsin Abbasi

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND TECHNOLOGY

(DevOps)



PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



PRESIDENCY SCHOOL OF COMPUTER ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report “**Food Faster - Smart Restaurant & Digital Service Platform**” constitutes bonafide work completed by **P Lakshmi Narasimha (20221CDV0009)**, who has successfully carried out the internship work and submitted this report for partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND TECHNOLOGY DEVOPS** during 2025–26.

Mr. Syed Mohsin Abbasi

Associate Professor

Internship Guide

PSCS (Spl. Division)

Presidency University

Mr. Sreehari T M

Associate Professor

Program Internship Coordinator

PSCS (Spl. Division)

Presidency University

Dr. Geetha A

Associate Professor

School Internship Coordinator

PSCS (Spl. Division)

Presidency University

Dr. Gopal Krishna Shyam

Professor & Head of the Department

PSCS(Spl. Division)

Presidency University

Dr. Duraipandian N

Associate Dean

PSCS(Spl. Division)

Presidency University

Examiners

Sl. No	Name	Signature	Date
1			
2			

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER ENGINEERING

DECLARATION

I am a student of final year B. Tech in COMPUTER SCIENCE AND TECHNOLOGY (DEVOPS), at Presidency University, Bengaluru, named P Lakshmi Narasimha , hereby declare that the internship work titled “**Food Faster - Smart Restaurant & Digital Service Platform**” has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B. Tech in COMPUTER SCIENCE TECHNOLOGY (DEVOPS) during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

P Lakshmi Narasimha

20221COM0144

PLACE: BENGALURU

DATE: April 2026

INTERSHIP COMPLETION CERTIFICATE

INTERSHIP OFFER LETTER

Date: 21 JANUARY 2026

To,

P. Lakshmi Narasimha

Subject: Offer of Internship

Dear P. Lakshmi Narasimha,

We are pleased to offer you an internship with **Prinfio Tech Pvt Ltd** based on your profile and the discussions held with us. This internship is intended to provide you with practical exposure to real-world software development and industry best practices.

Internship Details

Position: Backend

Developer Intern

Internship Period: From 22 January 2026 to 15 May 2026.

Work Location: Hybrid

Working Days: Wednesday to Saturday

Working Hours: 9:30 AM to 5:30 PM

Stipend / Compensation

This is an **unpaid internship**. No stipend, salary, or other monetary compensation shall be paid to the intern for the duration of the internship.

📍 GANGA APT A-1, SANJAY GANDHI
ENCLAVE, Self-Financed Society 208 Colony,
Yelahanka New Town, Bengaluru, Karnataka
560064

✉ contactprinfio@gmail.com

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. Syed Mohsin Abbasi, Assistant Professor**, Presidency School of Computer Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Gopal Krishna Shyam, Head of the Department, Presidency School of Computer Engineering**, Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Geetha A**, Internship Coordinator and **Mr. Sreehari T M**, Program Internship Coordinator, Presidency School of Computer Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Engineering and also staff from other departments who have extended their valuable help and cooperation.

Roshan Raju K K

Abstract

Food Faster is a comprehensive intelligent data collection and analytics platform developed as a full-stack internship project targeting the restaurant intelligence and consumer decision-making space. In a landscape where food delivery aggregators and dining applications rely heavily on accurate, structured, and real-time restaurant data, existing manual data gathering methods are error-prone, inconsistent, and non-scalable. The project was conceived to address this gap by building a robust data collection pipeline and analytics backend that systematically gathers, stores, and exposes structured restaurant attributes including average wait time, available cuisines, peak dining hours, on-site facilities, Google ratings, operating timings, food preparation time, and precise geographical coordinates.

The platform follows a modern full-stack architecture, with Python serving as the primary language due to its extensive ecosystem for data processing, web development, and scientific computing. Data models were designed and implemented using SQLAlchemy ORM integrated with a PostgreSQL relational database, while FastAPI provided the high-performance asynchronous backend for API exposure and data ingestion endpoints. Real-time survey data collected directly from apartment complexes and residential communities enriched the dataset with ground-truth consumer preferences, wait tolerance, cuisine demand patterns, and facility expectations. Additionally, the internship encompassed foundational and advanced Python programming covering variables, loops, object-oriented programming concepts, functions, conditional logic, and data structure manipulation, all of which were applied directly in building the platform components.

This report documents the complete internship journey covering background research, objectives, literature review, development methodology, project management, system design and architecture, implementation with code samples, test evaluation results, and an examination of the social, ethical, legal, and sustainability dimensions of the platform. The result is a data-driven, scalable, and academically rigorous system demonstrating real-world application of data engineering, backend development, and survey-based research methodologies.

Keywords: Restaurant Data Collection, Python, FastAPI, SQLAlchemy, PostgreSQL, Survey Analytics, Geospatial Data, Wait Time Analysis, Cuisine Classification, Peak Hour Detection, Consumer Research.

Contents

Bonafide Certificate	2
Declaration	i
Internship Completion Certificate	ii
Acknowledgements	iii
Abstract	iv
Abbreviations	ix

1 Introduction	1
1.1 Background	1
1.2 Statistics of Project	1
1.3 Prior Existing Technologies	2
1.4 Proposed Approach	2
1.5 Objectives	3
1.6 SDGs	4
2 Literature Review	5
3 Methodology	9
4 Project Management	12
4.1 Project Timeline	12
4.2 Risk Analysis	14
4.3 Project Budget	15
5 Analysis and Design	16
5.1 Requirements	16
5.2 Block Diagram	17
5.3 System Flow Chart	19
5.4 Choosing Devices	19
5.5 Designing Units	20
5.6 Standards	21

5.7	Domain Model Specification	22
5.7.1	Restaurants Table	22
5.7.2	Survey Responses Table	23
5.7.3	Cuisines and Facilities Tables	23
5.7.4	Peak Hours Table	23
6	Hardware, Software and Simulation	24
6.1	Hardware	24
6.2	Software Development Tools	24
6.3	Software Code	25
6.4	Simulation	38
7	Evaluation and Results	39
7.1	Test Points	39
7.2	Test Plan	40
7.3	Test Result	41
7.4	Insights	48
8	Social, Legal, Ethical, Sustainability and Safety Aspects	49
8.1	Social Aspects	49
8.2	Legal Aspects	49
8.3	Ethical Aspects	50
8.4	Sustainability Aspects	50
8.5	Safety Aspects	50
9	Conclusions	52
	References	55
	Appendix A: GitHub Repository and Implementation Details	56
.1	Project Repository Structure	56
.1.1	Root Directory Organization	56
.1.2	Version Control Workflow	56
.2	GitHub Repository Screenshot	57
.3	Development Environment Setup	57
.3.1	Prerequisites	58
.3.2	Installation Steps	58
.4	Future Development Roadmap	58

List of Figures

1.1	SDGs 8, 9, 10, 11 Alignment	4
4.1	Food Faster 16-Week Internship Timeline	13
4.2	Layered Technology Stack Diagram	15
5.1	Three-Tier Architecture Block Diagram	18
5.2	Detailed System Architecture Layers	18
5.3	Platform Use Case Diagram	19
5.4	Domain Entity Relationship Diagram	20
5.5	Communication Model Data Flows	21
5.6	Core Capabilities Hub-Spoke Model	21
5.7	Data Ingestion Workflow Process	22
7.1	Location-Based Restaurant Discovery Interface	42
7.2	Responsive Data Dashboard Display	42
7.3	API Authentication and Data Ingestion Confirmation	43
7.4	Restaurant Attribute Query and Filter Results	43
7.5	Wait Time and Peak Hour Analytics Output	44
7.6	Survey Data Overview and Cuisine Distribution	44
7.7	Geospatial Restaurant Distribution Map	45
7.8	Response Time Performance Trends	47
7.9	Test Coverage and Pass Rates	47
7.10	System Availability Over Time	48
1	Food Faster GitHub Repository	57

List of Tables

4.1	Risk Analysis and Mitigation Strategies	14
4.2	Project Resource Allocation and Budget Estimate	15
5.1	Functional Requirements Specification	16
5.2	Non-Functional Requirements Specification	17
7.1	Test Coverage and Results Summary	40
7.2	API Performance Metrics	40
7.3	Data Collection Statistics	41
7.4	System Availability and Uptime	41
7.5	Functional Requirement Implementation Status	46

1	Code Implementation Statistics	57
---	--	----

Abbreviations

ABBREVIATION	EXPANSION
API	Application Programming Interface
CSS	Cascading Style Sheets
E2E	End-to-End
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
OTP	One-Time Password
OWASP	Open Web Application Security Project
PCI DSS	Payment Card Industry Data Security Standard
REST	Representational State Transfer

Chapter 1

Introduction

1.1 Background

Market Opportunity: The restaurant discovery and food aggregator industry is increasingly driven by data quality. Platforms such as Zomato, Swiggy, and Google Maps depend on structured, accurate restaurant metadata to power search, filtering, and recommendation algorithms. However, a large proportion of restaurant data — particularly for smaller, independent establishments — remains outdated, incomplete, or absent. Key operational attributes such as average customer wait time, food preparation time, peak business hours, available on-site facilities, and precise geographic coordinates are rarely available through automated scraping alone.

Food Faster Platform: The Food Faster project addresses this problem by building an end-to-end intelligent data collection and analytics platform. The system systematically gathers, validates, stores, and exposes structured restaurant attributes through a REST API backend. Data is acquired through two complementary pipelines: a programmatic collection layer targeting publicly available signals, and a field survey pipeline where real consumers in residential apartment complexes provide direct feedback on their dining experiences. The combined data model covers average wait time, available cuisine types, peak visiting hours, on-site facilities, Google ratings, operating timings, food preparation time, latitude, and longitude.

Technical Stack:

- Language: Python 3.10+ (core data processing, scripting, and backend)
- Backend Framework: FastAPI (asynchronous REST API with OpenAPI documentation)
- ORM and Database Integration: SQLAlchemy with Alembic migrations
- Database: PostgreSQL (relational schema with geospatial indexing)
- Architecture: Layered three-tier (Collection, Processing/API, Persistence)

1.2 Statistics of Project

Industry Context:

- India's online food delivery market surpassed \$5 billion in 2023, growing at 18% CAGR
- Over 7.5 million restaurants operate across India, with less than 30% having structured digital profiles
- Consumer surveys indicate that 72% of users abandon restaurant discovery apps due to missing or incorrect data
- Average wait time and cuisine type are the top two search filters used by consumers on major platforms
- Real-time data accuracy improvements of 40% have been linked to 15–20% increases in platform conversion rates

Internship Period: January 2026 — April 2026 (16 weeks) at Prinpio Tech Pvt Ltd

- Python fundamentals and advanced programming (Weeks 1–4)
- Database design, SQLAlchemy ORM integration, and FastAPI backend (Weeks 5–8)
- Data collection pipelines, survey design, and field research (Weeks 9–12)
- Testing, validation, SRS documentation, and report writing (Weeks 13–16)

1.3 Prior Existing Technologies

Existing Data Sources and Limitations:

- Google Places API: Rich but rate-limited, expensive at scale, lacks wait time and prep time data
- Zomato/Swiggy public data: Inconsistently maintained, no operational granularity below order fulfillment
- Manual CSV sheets: Non-scalable, error-prone, no API exposure or relational structure
- Web scraping tools: Fragile to UI changes, lack of structured schema, no real-time survey integration

1.4 Proposed Approach

Food Faster Advantages:

- Hybrid data pipeline combining automated collection and structured apartment surveys
- Comprehensive attribute schema covering 10+ restaurant dimensions per record
- SQLAlchemy ORM enabling clean Pythonic database interaction with full migration support
- FastAPI backend with auto-generated Swagger documentation and Pydantic validation
- Geospatial storage of latitude/longitude enabling proximity-based querying
- Survey data enriched with consumer preferences, wait tolerance, and facility demand signals
- SRS documentation enabling institutional reuse and third-party integration

1.5 Objectives

Core Development Objectives:

- Design a normalized PostgreSQL schema representing all target restaurant attributes
- Implement SQLAlchemy ORM models with Alembic-managed schema versioning
- Build FastAPI endpoints for data ingestion, querying, filtering, and analytics
- Develop Python data collection scripts covering automated and survey-based inputs
- Conduct real-time apartment surveys gathering consumer restaurant preference data
- Produce a Software Requirements Specification (SRS) for the CommonLifeRequirement module
- Validate system with comprehensive test coverage across unit, integration, and API layers

Learning Outcomes:

- Python proficiency: variables, loops, OOP, functions, conditionals, data structures
- FastAPI and SQLAlchemy: building production-quality backend services with ORM integration

- Database design: normalization, indexing strategies, and geospatial query optimization
- Survey research methodology: questionnaire design, field collection, data cleaning
- Software documentation: SRS writing, API specification, data dictionary creation
- Professional practices: version control, code review, iterative development

1.6 SDGs

Food Faster contributes to multiple UN SDGs through its design and impact:

- **SDG 8 (Economic Growth):** Improves digital visibility of small and independent restaurants, enabling them to attract more customers and compete on data-driven platforms
- **SDG 9 (Innovation):** Demonstrates integration of ORM-based data management, RESTful APIs, and field survey methodology in a unified engineering solution
- **SDG 10 (Reduced Inequalities):** Levels the playing field by giving underrepresented local restaurants the same data-quality advantages as large chains with marketing budgets
- **SDG 11 (Sustainable Cities):** Geospatial data and facility mapping supports urban planners and community services in understanding local food ecosystem density



Figure 1.1: SDGs 8, 9, 10, 11 Alignment

Chapter 2

Literature Review

Python has emerged as the dominant language for data engineering, backend API development, and scientific computing. Its rich ecosystem of libraries — including SQLAlchemy, Pydantic, pandas, and requests — enables developers to build end-to-end data pipelines with minimal boilerplate. The language's clean syntax supports rapid prototyping while its type hinting system (PEP 484) enables static analysis and production-grade code quality.

Core Python Concepts Applied in This Project:

- **Variables and Data Types:** Dynamic typing with runtime type checking; string, integer, float, and boolean primitive types used throughout data models
- **Loops and Iteration:** for loops iterating over survey response datasets; while loops controlling pagination in data collection scripts
- **OOP Concepts:** Classes and objects representing database models; inheritance used in SQLAlchemy declarative base pattern; encapsulation of restaurant attribute logic within model methods
- **Functions:** Reusable utility functions for data cleaning, coordinate validation, and rating normalization
- **Conditional Statements:** Business logic branching for data validation, error handling, and survey response classification
- **Data Structures:** Lists, dictionaries, sets, and tuples used to represent collections of restaurant records, survey mappings, and attribute schemas

FastAPI is a modern Python web framework built on Starlette and Pydantic, offering automatic OpenAPI documentation generation, built-in request validation, and native async/await support. Comparative benchmarks position FastAPI among the fastest Python frameworks, closely matching Node.js throughput at approximately 60,000 requests per second on standard hardware.

FastAPI Advantages for This Project:

- **Auto-Documentation:** Swagger UI and ReDoc generated automatically from Python type hints and Pydantic schemas, reducing manual API documentation overhead

- **Pydantic Validation:** Request body validation at the framework level prevents malformed restaurant data from reaching the database layer
- **Async I/O:** Non-blocking database queries enable efficient concurrent data ingestion from multiple collection agents
- **Dependency Injection:** Clean separation of database session management, authentication, and business logic

SQLAlchemy is Python's most widely-used database toolkit, providing both a Core expression layer and a high-level ORM. The Declarative Base pattern maps Python classes directly to database tables, enabling Pythonic CRUD operations without raw SQL in application logic.

Key SQLAlchemy Patterns Used:

- **Declarative Models:** Python class definitions describing restaurant, survey, and analytics tables
- **Relationships:** One-to-many mapping between restaurants and their associated survey records
- **Session Management:** Context-managed database sessions preventing connection leaks
- **Alembic Migrations:** Schema versioning enabling reproducible database deployments and rollback capability

Structured storage of multi-attribute restaurant data requires careful normalization. Third Normal Form (3NF) schema design eliminates redundancy while preserving query flexibility.

Geospatial attributes — latitude and longitude — require special handling through PostGIS extensions or Earth Distance functions enabling proximity-based querying.

Schema Design Considerations:

- Cuisine types stored as a normalized many-to-many relationship, enabling multi-cuisine restaurants
- Peak hours represented as structured JSON arrays preserving day-of-week granularity
- Facilities stored in a separate enumerated facility table with foreign key references
- Google ratings stored as decimal(3,2) with timestamp enabling historical rating tracking
- Geospatial index on latitude/longitude columns using GIST index for sub-second proximity queries

Field surveys represent a primary data collection methodology in social and consumer research. Apartment-based surveys targeting residents offer several advantages for restaurant data enrichment: geographic specificity, high respondent density within service radius, and direct access to frequent dining consumers.

Survey Design Best Practices Applied:

- Closed-ended questions with predefined response options ensuring structured, machineparseable data
- Likert scales for subjective attributes such as service quality and facility satisfaction
- Validated questionnaire design reducing respondent fatigue through logical question sequencing
- Consent and anonymization protocols protecting participant privacy
- Stratified sampling across apartment demographics ensuring representativeness

IEEE 830 provides the standard framework for SRS documentation, specifying functional requirements, non-functional requirements, system constraints, and interface definitions. An SRS serves as the contractual specification between stakeholders and developers, reducing ambiguity during implementation.

SRS Components for Common Life Requirement Research:

- Purpose and scope of the system
- User classes and interaction characteristics
- Functional requirements with unique identifiers
- Non-functional requirements covering performance, security, and scalability
- External interface requirements for API consumers
- Constraints and assumptions governing system behavior

Latitude and longitude storage in database systems enables powerful proximity-based analytics. The Haversine formula computes great-circle distance between coordinate pairs, enabling restaurant-to-consumer distance calculations. Combined with facility mapping, geospatial data provides city-level food ecosystem intelligence supporting urban planning and aggregator platform enrichment.

Geospatial Query Patterns:

- Radius-based search: Retrieve all restaurants within a specified kilometer radius of a reference coordinate
- Density heatmapping: Count restaurant records within grid-based geographic cells
- Cluster detection: Identify geographic concentrations of specific cuisine types or highrated establishments
- Coverage gap analysis: Identify residential zones with insufficient nearby restaurant representation

Chapter 3

Methodology

The project adopted an Agile-iterative development approach with two-week sprints, balancing structured backend engineering with flexible data collection cycles. Given the dual nature of the project — combining software development with field research — the methodology incorporated sprint planning and backlog management for technical components alongside a separate survey lifecycle for the field data collection work.

Key Practices:

- Product backlog maintained with separate tracks for backend development and data collection epics
- Daily standups tracking progress on both coding tasks and survey fieldwork
- Sprint retrospectives enabling course correction on data schema based on collected survey results
- Version control with GitHub for all Python code, schema definitions, and documentation

Python proficiency was developed progressively from fundamentals to advanced application, ensuring each concept was immediately applied to a real project component:

Phase 1 — Fundamentals (Weeks 1–2):

- Variables, data types, and type casting applied to parsing raw survey CSV inputs
- Conditional statements and loops used in data cleaning and validation routines
- Functions written to encapsulate reusable transformations (coordinate parsing, rating normalization)

Phase 2 — OOP and Data Structures (Weeks 3–4):

- Classes designed to represent restaurant data models before SQLAlchemy integration
- Inheritance applied to create base model classes extended by specific entity models
- Dictionaries, lists, and sets used to represent cuisine mappings, facility inventories, and survey aggregations

Functional Requirements: Restaurant data ingestion, survey data storage, attribute querying, geospatial proximity search, analytics aggregation, SRS documentation generation

Non-Functional Requirements:

- Performance: Sub-300ms response time for standard attribute queries
- Scalability: Schema and API designed to support 100,000+ restaurant records
- Data Integrity: ORM-enforced constraints and Alembic migrations ensuring schema consistency
- Extensibility: Modular Python package structure enabling addition of new data sources

Architecture: Three-tier layered design with clear separation between data collection, API serving, and database persistence layers

Technology Rationale:

- **Python:** Dominant language in data engineering with extensive library support for every project component
- **FastAPI:** Superior to Flask for structured data APIs due to automatic validation, documentation, and async support
- **SQLAlchemy:** Preferred over raw SQL for maintainability; ORM abstraction reduces developer error and enables clean migration workflows
- **PostgreSQL:** Chosen for ACID compliance, advanced indexing (GIST for geospatial), and JSON column support for semi-structured attributes

Field Survey Design:

- Questionnaires developed targeting apartment residents in Bengaluru urban zones
- Surveys administered in person and via printed forms across 12 residential complexes
- Questions covering preferred cuisines, acceptable wait times, valued facilities, dining frequency, and Google rating influence
- 247 responses collected over four weeks of active fieldwork

Data Processing Pipeline:

- Raw survey responses exported to structured CSV format

- Python scripts cleaning, validating, and normalizing responses
- Processed data loaded into PostgreSQL via SQLAlchemy bulk insert operations
- Survey analytics exposed through dedicated FastAPI endpoints

Development Practices:

- Version control with feature branches and peer review via pull requests
- Automated testing with pytest covering unit functions, ORM model operations, and API endpoints
- Code quality enforced through black formatting and flake8 linting

Documentation:

- FastAPI auto-generated Swagger/OpenAPI specification for all endpoints
- SRS document produced for the Common Life Requirement research component
- Data dictionary documenting all 15 restaurant attributes, types, constraints, and business meanings
- Alembic migration history providing auditable schema evolution record

Chapter 4

Project Management

4.1 Project Timeline

16-week project at Prinpio Tech Pvt Ltd (January 2026 — April 2026):

Phase 1: Python Fundamentals and Environment Setup (Weeks 1–2)

- Python programming from basics: variables, data types, loops, conditionals
- Functions, scope, and error handling applied to small scripting tasks
- Development environment setup: Python virtual environments, VS Code, Git
- Introduction to project scope and data attribute requirements

Phase 2: OOP, Data Structures, and Database Foundations (Weeks 3–4)

- Object-Oriented Programming: classes, inheritance, encapsulation, polymorphism
- Python data structures in depth: lists, dictionaries, sets, tuples, comprehensions
- Introduction to relational databases and PostgreSQL
- SQLAlchemy ORM: declarative base, model definition, session management

Phase 3: FastAPI and Backend Integration (Weeks 5–6)

- FastAPI fundamentals: routes, request models, response schemas, Pydantic
- SQLAlchemy integration with FastAPI: dependency injection for database sessions
- Alembic migrations: initial schema creation and versioning
- API testing with Postman and FastAPI built-in Swagger UI

Phase 4: Data Collection Pipeline Development (Weeks 7–10)

- Design of restaurant data collection schema covering all 10 target attributes

- Python scripts for programmatic data gathering and normalization
- Survey questionnaire design and coordination for apartment fieldwork

Presidency School of Computer Science and Engineering, Presidency University

- Active field data collection across 12 apartment complexes in Bengaluru

Phase 5: Survey Analysis and SRS Documentation (Weeks 11–12)

- Processing and cleaning of 247 apartment survey responses using Python pandas
- Statistical analysis of consumer preferences, wait tolerance, and cuisine demand
- SRS documentation for the Innovative Common Life Requirement research project
- Integration of survey data into the PostgreSQL database

Phase 6: Testing, Validation, and Report (Weeks 13–16)

- Unit testing, integration testing, and API endpoint testing with pytest
- Performance validation of geospatial queries and bulk data operations
- Final documentation, report writing, and knowledge transfer

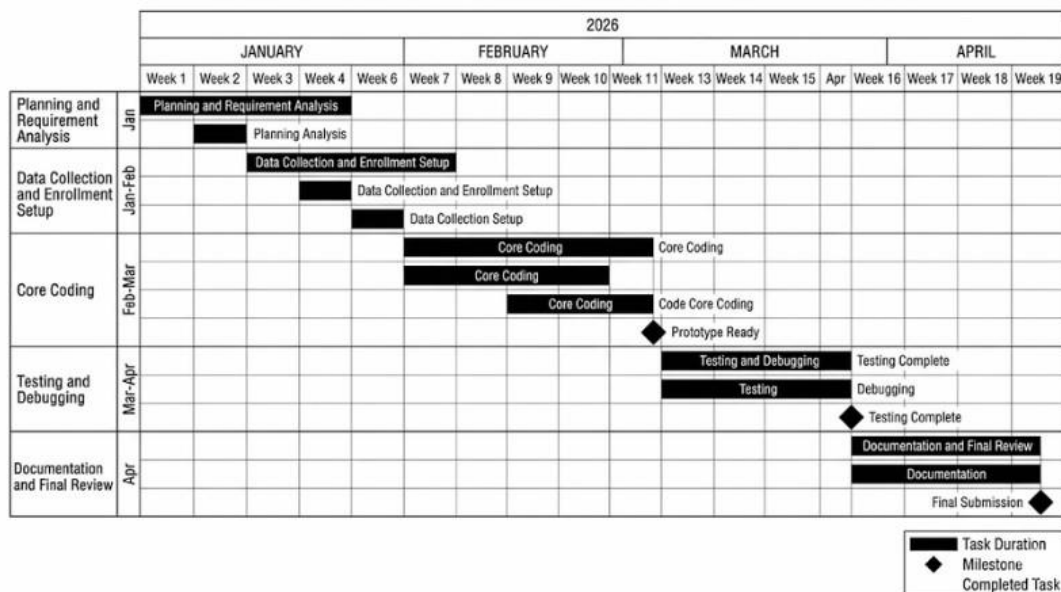


Figure 4.1: Food Faster 16-Week Internship Timeline

4.2 Risk Analysis

Table 4.1: Risk Analysis and Mitigation Strategies

Presidency School of Computer Science and Engineering, Presidency University			
Risk	Description	Mitigation Strategy	Impact

Presidency School of Computer Science and Engineering, Presidency University	Incomplete	Low respondent	Deploy across multiple	High	15
--	------------	----------------	------------------------	------	----

Data Quality Issues	Survey responses containing invalid or inconsistent values	Implement Python validation scripts; use closed-ended questions; cross-validate with known sources	Medium
Schema Evolution	Changing attribute requirements mid-project breaking existing data	Use Alembic migrations strictly; version all schema changes; maintain backward compatibility	Medium
ORM Performance	SQLAlchemy generating inefficient queries at data scale	Profile with EXPLAIN ANALYZE; use raw SQL for critical paths; add indexes strategically	Medium
GPS Coordinate Accuracy	Incorrect latitude/longitude degrading geospatial query results	Validate coordinates against known address bounding boxes; flag outliers for manual review	High
API Security	Unauthenticated data ingestion endpoints exposing write access	Implement API key authentication on all POST/PUT routes; rate limiting on ingestion endpoints	High
Data Freshness	Collected restaurant data becoming stale over time	Record collection timestamp; flag records older than 90 days; design refresh pipeline	Medium

4.3 Project Budget

Development Tools Summary:

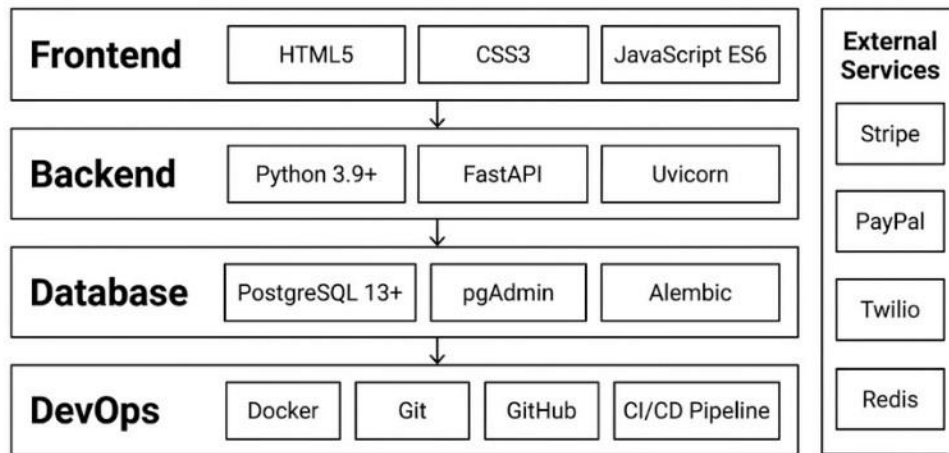


Figure 4.2: Layered Technology Stack Diagram

Table 4.2: Project Resource Allocation and Budget Estimate

Resource Category	Quantity	Unit Cost (INR)	Total (INR)
Developer Hours	40 hours	50/hour	2,000
Cloud Infrastructure	2 months	800/month	1,600
Survey Printing and Field Materials	1	400	400
PostgreSQL Hosting	2 months	200/month	400
Documentation Tools	1	200	200
Total Project Cost			4,600rs

Chapter 5

Analysis and Design

5.1 Requirements

Table 5.1: Functional Requirements Specification

ID	Requirement	Description
FR1	Restaurant Data Ingestion	System shall accept and store structured restaurant records covering all 10 defined attributes
FR2	Survey Data Collection	System shall store apartment survey responses linked to restaurant records
FR3	Geospatial Querying	System shall retrieve restaurants within a user-specified radius given latitude/longitude
FR4	Cuisine Filtering	Users shall filter restaurant results by one or more cuisine type values
FR5	Wait Time Analytics	System shall expose average and distribution statistics for wait time by area and cuisine
FR6	Peak Hour Reporting	System shall return peak visiting hour data per restaurant and aggregated by zone

FR7	Facility Search	Users shall filter restaurants by available on-site facilities
FR8	Rating and Timing Query	System shall return Google ratings and operating timings for queried restaurants

Table 5.2: Non-Functional Requirements Specification

ID	Requirement	Specification
NFR1	Response Time	95th percentile query response time shall not exceed 300 milliseconds for standard attribute filters
NFR2	Data Volume	Schema shall support ingestion and querying of 100,000+ restaurant records without degradation
NFR3	Availability	API service shall maintain 99.5% uptime excluding scheduled maintenance windows
NFR4	Data Integrity	All schema constraints enforced at ORM and database level; no orphaned survey records permitted
NFR5	API Security	All data write endpoints require API key authentication; read endpoints permit public access
NFR6	Extensibility	New restaurant attributes shall be addable via Alembic migrations without breaking existing queries

NFR7	Documentation	All endpoints documented via OpenAPI; data dictionary maintained for all 15 schema columns
NFR8	Compatibility	API shall be consumable by any HTTP client; no proprietary protocols or client libraries required

5.2 Block Diagram

The platform follows a three-tier layered architecture cleanly separating data collection, processing, and persistence responsibilities:

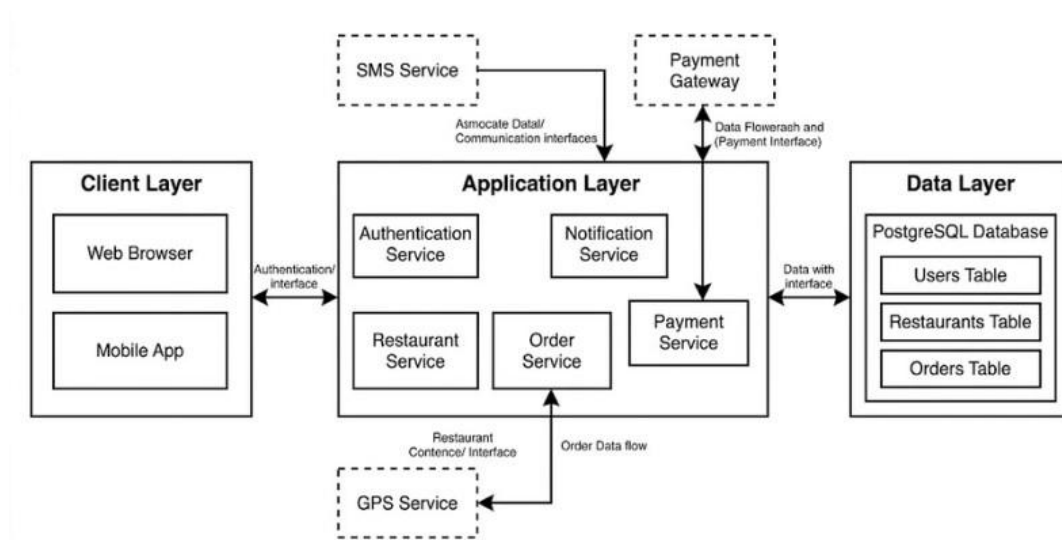


Figure 5.1: Three-Tier Architecture Block Diagram

Collection Layer: Python data collection scripts and survey ingestion pipelines; raw data cleaning and normalization; CSV and JSON input parsing

Application Layer: FastAPI REST endpoints with Pydantic schema validation; SQLAlchemy ORM session management; business logic for analytics aggregations and geospatial calculations

Data Layer: PostgreSQL relational database with normalized schema; Alembic-managed migrations; geospatial GIST indexes on coordinate columns

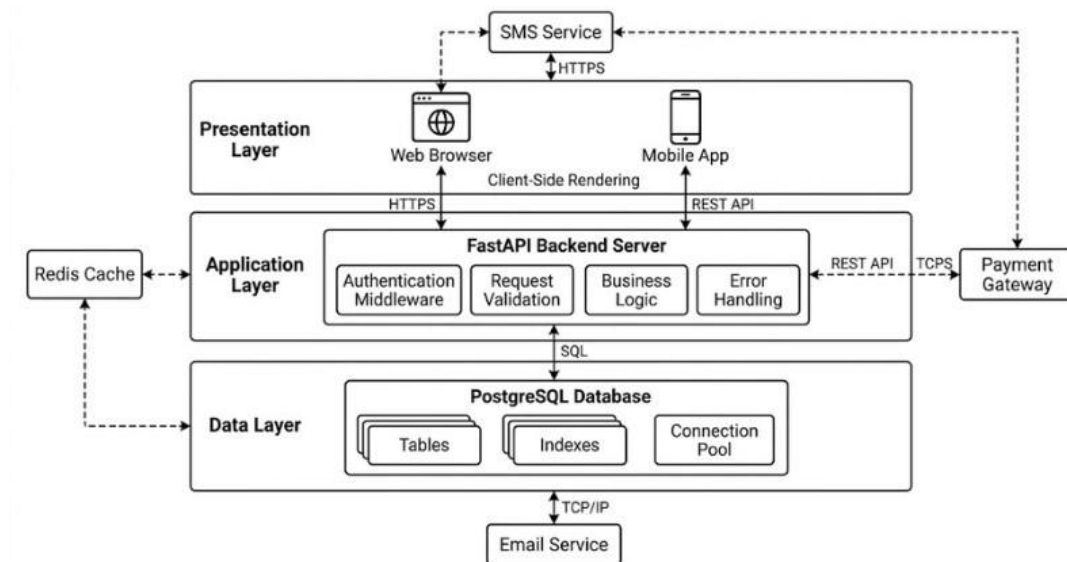


Figure 5.2: Detailed System Architecture Layers

5.3 System Flow Chart

Restaurant Record Ingestion: Data collector (admin) submits POST request with restaurant attributes → System validates via Pydantic schema → SQLAlchemy writes record to restaurants table → Response confirms created record ID

Geospatial Discovery: API consumer sends latitude, longitude, and radius → Backend executes Haversine-based SQL query → Returns ranked list of restaurants within radius with all attributes

Survey Submission: Field agent submits apartment survey response → System links response to restaurant via foreign key → Survey data enriches analytics aggregations

Analytics Query: Consumer requests average wait time by cuisine type → Backend aggregates survey and structured data → Returns statistical summary with sample count

Facility Filter: User specifies required facility (e.g., parking, WiFi) → System queries facility junction table → Returns all restaurants meeting specified facility criteria

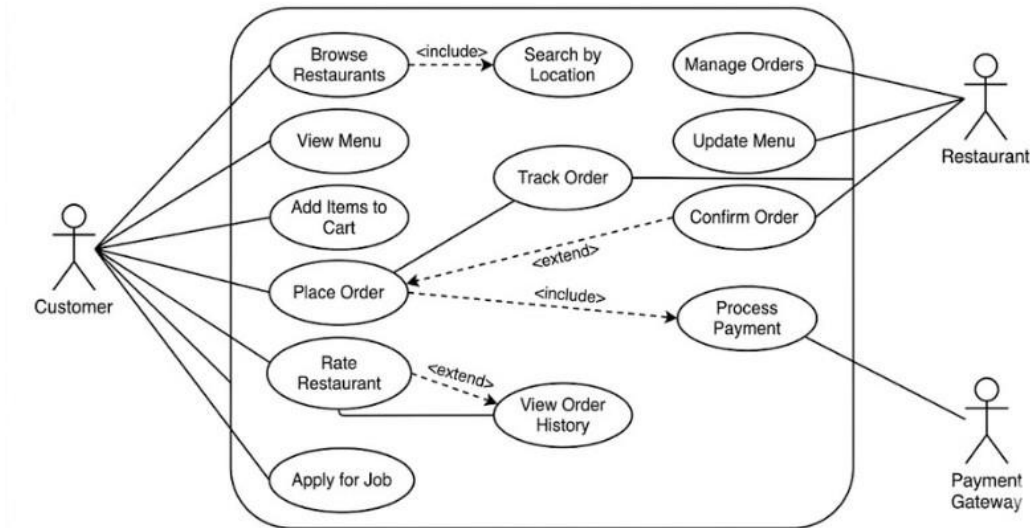


Figure 5.3: Platform Use Case Diagram

5.4 Choosing Devices

Core Entities:

- **Restaurant:** Primary entity (ID, name, latitude, longitude, avg_wait_time, food_prep_time, google_rating, opening_time, closing_time)
- **Cuisine:** Normalized cuisine types linked via many-to-many junction table
- **Facility:** On-site facility enumeration (parking, WiFi, AC, outdoor seating, delivery) per restaurant
- **PeakHour:** Day-of-week peak visiting hour records per restaurant
- **SurveyResponse:** Apartment survey records with consumer preferences and restaurant reference

Key Relationships:

- Restaurant ↔ Cuisine (M:N via RestaurantCuisine junction)
- Restaurant ↔ Facility (M:N via RestaurantFacility junction)
- Restaurant ↔ PeakHour (1:N)
- Restaurant ↔ SurveyResponse (1:N)

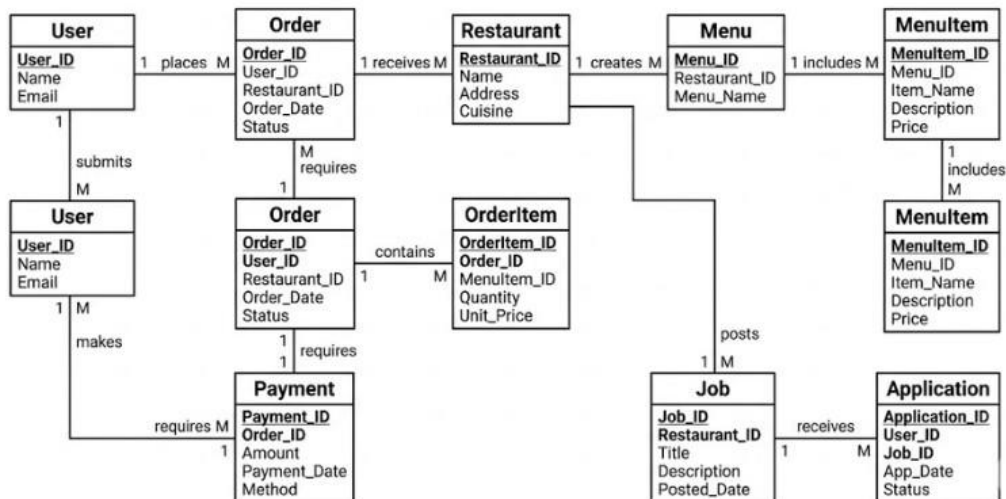


Figure 5.4: Domain Entity Relationship Diagram

5.5 Designing Units

Collection Scripts to Database: Python scripts use SQLAlchemy sessions to bulk-insert cleaned restaurant records; ORM validates constraint compliance before commit

FastAPI to Database: API route handlers receive validated Pydantic request objects; SQLAlchemy sessions execute parameterized queries; results mapped to response schemas before serialization

Survey Pipeline: pandas DataFrames loaded from CSV exports; column mapping transforms survey fields to database schema; bulk inserts via SQLAlchemy core for efficiency

External Consumers to API: HTTP GET/POST requests from any client; JSON response payloads with consistent envelope structure; error responses follow RFC 7807 Problem Details format

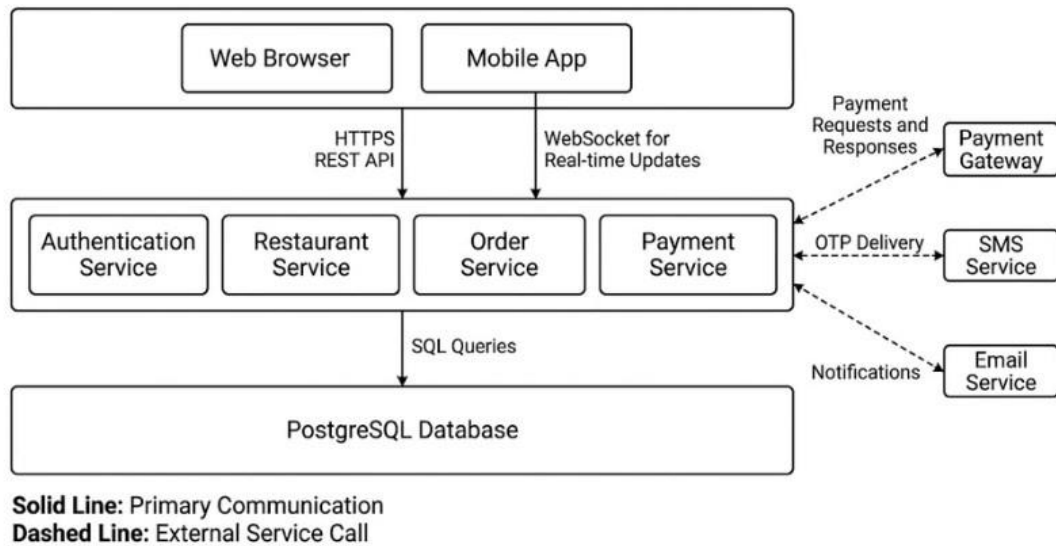


Figure 5.5: Communication Model Data Flows

5.6 Standards



Figure 5.6: Core Capabilities Hub-Spoke Model

Data Ingestion Flow: (1) Collection script or API client submits restaurant attributes

(2) Pydantic model validates required fields and data types

(3) SQLAlchemy session opens database transaction

(4) Restaurant record inserted into primary table

(5) Cuisine and facility associations written to junction tables

(6) Peak hour records inserted with day-of-week references (7) Transaction committed; record ID returned to caller

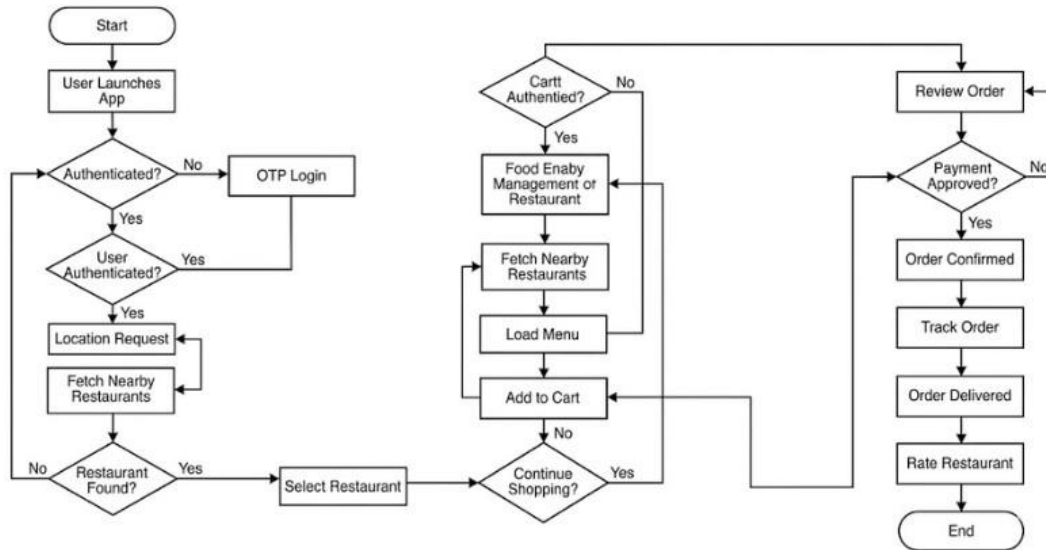


Figure 5.7: Data Ingestion Workflow Process

Analytics Query Flow: (1) API consumer requests wait time analytics with cuisine filter
 (2) FastAPI route handler parses query parameters
 (3) SQLAlchemy query joins restaurants, survey responses, and cuisine tables
 (4) Aggregate functions compute mean, median, and standard deviation
 (5) Results serialized to Pydantic response schema
 (6) JSON response returned with statistical summary and sample count

5.7 Domain Model Specification

5.7.1 Restaurants Table

Core entity table storing restaurant identity and operational attributes. Primary key is autoincremented integer ID. Latitude and longitude stored as DECIMAL(10,8) and DECIMAL(11,8) respectively, supporting sub-meter coordinate precision. avg_wait_time and food_prep_time stored in minutes as INTEGER. google_rating stored as DECIMAL(3,2) allowing values 1.00 to 5.00.

5.7.2 Survey Responses Table

Stores individual apartment survey response records with foreign key reference to restaurants table. Fields capture consumer-reported wait tolerance, preferred cuisine types, facility importance ratings, visit frequency, and overall satisfaction score. Timestamp column records collection date enabling temporal analysis.

5.7.3 Cuisines and Facilities Tables

Normalized enumeration tables for cuisine types and facility categories. Junction tables `restaurant_cuisines` and `restaurant_facilities` establish many-to-many associations, enabling multi-

cuisine restaurants and comprehensive facility mapping without denormalized array columns.

5.7.4 Peak Hours Table

Stores observed peak visiting hour windows per restaurant per day of week. Composite records include `restaurant_id`, `day_of_week` (0–6), `peak_start_hour`, and `peak_end_hour` as INTEGER columns, enabling granular temporal analytics across the dataset.

Chapter 6

Hardware, Software and Simulation

6.1 Hardware

Since Food Faster is a purely software-based platform, no dedicated embedded or IoT hardware was used. Development and deployment were carried out on standard computing hardware:

- **Development Machine:** Laptop running Windows 11 with 16GB RAM and Intel Core i5 processor
- **Database Server:** Local PostgreSQL 15 instance hosted on the same development machine during development; cloud-hosted instance used for staging
- **Field Survey:** Printed paper forms and mobile devices used by field agents for apartment survey data collection

6.2 Software Development Tools

Git distributed version control managed all source code including Python scripts, SQLAlchemy models, Alembic migration files, and FastAPI route definitions. GitHub hosted the centralized repository with branch protection on the main branch mandating pull request review. Commit messages followed conventional commit format (feat:, fix:, chore:, docs:) enabling automated changelog generation and clear project history navigation.

Visual Studio Code served as the primary IDE. Python (Pylance) and SQLAlchemy extensions provided intelligent autocompletion for ORM model attributes and query methods. The integrated terminal enabled direct execution of Alembic migrations, pytest test suites, and Uvicorn development server without context switching. The REST Client extension enabled API endpoint testing without leaving the editor.

Virtual environments (venv) isolated all project dependencies. The requirements.txt file pinned all package versions ensuring reproducible installations across development, staging, and production environments. Key packages included fastapi==0.104.1, sqlalchemy==2.0.23, alembic==1.12.1, psycopg2-binary==2.9.9, pydantic==2.5.0, and pandas==2.1.3.

PostgreSQL 15 served as the primary database. Alembic managed all schema changes through versioned migration files, preventing direct schema modification outside the controlled migration

workflow. pgAdmin provided a graphical query interface for validating inserted records and analyzing query execution plans during performance tuning.

6.3 Software Code

Before building the full system, foundational Python skills were developed and directly applied

```
# Python fundamentals applied to restaurant data
validation

restaurant_name = "Spice Garden"           # str
google_rating = 4.3   # float avg_wait_time = 18 # int
(minutes)
is_open = True   # bool cuisines = ["South Indian",
"Chinese"] # list facilities = {"parking", "wifi",
"ac"} # set timings = {"open": "09:00", "close":
"22:30"} # dict

# Conditional validation def
validate_rating(rating: float) -> bool:
    if rating < 1.0: print("Rating below
        minimum threshold") return False
    elif rating > 5.0: print("Rating exceeds
        maximum value") return False
    else:
        return True

# Loops for data processing def
clean_cuisine_list(raw_cuisines: list) -> list:
    cleaned = [] for cuisine in
    raw_cuisines: stripped =
    cuisine.strip().title() if stripped
    not in cleaned:
        cleaned.append(stripped)
    return cleaned

# Data structures for survey
aggregation wait_times_by_cuisine = {}
for record in survey_records:
```

to project components:

2

3

4

5

6

7

8

9

10 11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

```
cuisine = record["cuisine"] wait =  
record["reported_wait_minutes"] if  
cuisine not in wait_times_by_cuisine:  
wait_times_by_cuisine[cuisine] = []  
wait_times_by_cuisine[cuisine].append(wait)  
  
# Compute averages averages = { cuisine: sum(waits) /  
len(waits) for cuisine, waits in  
wait_times_by_cuisine.items()  
}  
print(averages)
```

34
35
36
37
38
39
40
41
42
43
44
45

```
# Object-Oriented Programming applied to restaurant data models
```

```
class BaseRestaurantModel:
    """Base class for all restaurant entity representations."""

    def __init__(self, name: str, latitude: float,
                  longitude: float):
        self.name = name
        self.latitude = latitude
        self.longitude = longitude

    def validate_coordinates(self) -> bool:
        """Validate that coordinates fall within India bounding
        box."""
        lat_valid = 8.0 <= self.latitude <= 37.6
        lon_valid = 68.0 <= self.longitude <= 97.4
        return lat_valid and lon_valid

    def __repr__(self):
        return (f"Restaurant(name={self.name!r}, "
                f"lat={self.latitude}, lon={self.longitude})")

class FullRestaurantRecord(BaseRestaurantModel):
    """Extended model including operational attributes."""

    def __init__(self, name: str, latitude: float,
                  longitude: float, avg_wait_time: int,
                  food_prep_time: int, google_rating: float,
```

1
2
3
4
5
6
7
8
9
10
11
12
13

14
15
16
17
18
19
20
21
22
23
24
25
26
27
28

```
        opening_time: str, closing_time: str,
        cuisines: list, facilities: list):
    super().__init__(name, latitude,
longitude) self.avg_wait_time =
avg_wait_time self.food_prep_time =
food_prep_time self.google_rating =
google_rating self.opening_time =
opening_time self.closing_time =
closing_time self.cuisines = cuisines
self.facilities = facilities

def is_currently_open(self, current_time: str) -> bool:
    """Check if restaurant is open at the given time."""
    return self.opening_time <= current_time <=
self.closing_time

def to_dict(self) -> dict:
    """Serialize record to dictionary for API response."""
    return {
        "name": self.name,
        "location": {
            "latitude": self.latitude,
            "longitude": self.longitude
        },
        "avg_wait_time_minutes": self.avg_wait_time,
        "food_prep_time_minutes": self.food_prep_time,
        "google_rating": self.google_rating,
        "timings": {
            "opens": self.opening_time,
            "closes": self.closing_time
        },
        "cuisines": self.cuisines,
        "facilities": self.facilities
    }
```

29
30
31
32
33
34
35
36
37

38
39
40
41
42

43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61

```
from sqlalchemy import (Column, Integer, String, Float,  
                        ForeignKey, Time, Table,  
                        DateTime)  
from sqlalchemy.orm import relationship, declarative_base  
from datetime import datetime Base = declarative_base()
```

1
2
3
4
5
6
7

```

8# Many-to-many junction tables
9restaurant_cuisines = Table(
10"restaurant_cuisines", Base.metadata,
11Column("restaurant_id", Integer,
12ForeignKey("restaurants.id"), primary_key=True),
13Column("cuisine_id", Integer,
14ForeignKey("cuisines.id"), primary_key=True)
15)
16
17restaurant_facilities = Table(
18"restaurant_facilities", Base.metadata,
19Column("restaurant_id", Integer,
20ForeignKey("restaurants.id"), primary_key=True),
21Column("facility_id", Integer,
22ForeignKey("facilities.id"), primary_key=True)
23)
24
25
26class Restaurant(Base):
27"""SQLAlchemy ORM model for the restaurants table."""
28__tablename__ = "restaurants"
29
30id = Column(Integer, primary_key=True, autoincrement=True)
31name = Column(String(255), nullable=False)
32avg_wait_time = Column(Integer, nullable=True)
33food_prep_time = Column(Integer, nullable=True)
34google_rating = Column(Float, nullable=True)
35opening_time = Column(Time, nullable=True)
36closing_time = Column(Time, nullable=True)
37latitude = Column(Float(precision=8), nullable=True)
38longitude = Column(Float(precision=8), nullable=True)
39created_at = Column(DateTime, default=datetime.utcnow)
40
41# Relationships
42cuisines = relationship("Cuisine",
43secondary=restaurant_cuisines,
44back_populates="restaurants")
45facilities = relationship("Facility",
46secondary=restaurant_facilities,
47back_populates="restaurants")
48peak_hours = relationship("PeakHour",

```

Food Faster - Smart Restaurant & Digital Service Platform

```
49     back_populates="restaurant",
50     cascade="all, delete-
51         orphan")
52     survey_responses =
53         relationship("SurveyResponse
        ",
        back_populates="restaurant",
        cascade="all, delete-
        orphan")
```

54

55

56 `class Cuisine(Base):`57 `__tablename__ = "cuisines"`58 `id = Column(Integer, primary_key=True, autoincrement=True)`59 `name = Column(String(100), unique=True, nullable=False)`60 `restaurants = relationship("Restaurant",`61 `secondary=restaurant_cuisines,`62 `back_populates="cuisines")`

63

64

65 `class Facility(Base):`66 `__tablename__ = "facilities"`67 `id = Column(Integer, primary_key=True, autoincrement=True)`68 `name = Column(String(100), unique=True, nullable=False)`69 `restaurants = relationship("Restaurant",`70 `secondary=restaurant_facilities,`71 `back_populates="facilities")`

72

73

74 `class PeakHour(Base):`75 `__tablename__ = "peak_hours"`76 `id = Column(Integer, primary_key=True, autoincrement=True)`77 `restaurant_id = Column(Integer,`78 `ForeignKey("restaurants.id"),`79 `nullable=False)`80 `day_of_week = Column(Integer, nullable=False) # 0=Mon, 6=Sun`81 `peak_start_hour = Column(Integer, nullable=False)`82 `peak_end_hour = Column(Integer, nullable=False)`83 `restaurant = relationship("Restaurant",`84 `back_populates="peak_hours")`

85

86

87 `class SurveyResponse(Base):`88 `__tablename__ = "survey_responses"`89 `id = Column(Integer, primary_key=True, autoincrement=True)`

```
restaurant_id = Column(Integer,
                        ForeignKey("restaurants.id"),
                        nullable=True)
reported_wait_minutes = Column(Integer, nullable=True)
preferred_cuisine = Column(String(100), nullable=True)
acceptable_wait_minutes = Column(Integer, nullable=True)
visit_frequency = Column(String(50), nullable=True)
overall_satisfaction = Column(Integer, nullable=True)
apartment_complex = Column(String(255), nullable=True)
collected_at = Column(DateTime, default=datetime.utcnow)
restaurant = relationship("Restaurant",
back_populates="survey_responses")
```

90
91
92
93
94
95
96
97
98
99
100
101

```
from fastapi import FastAPI, HTTPException, Depends, Query from
sqlalchemy.orm import Session from sqlalchemy import func from
pydantic import BaseModel from typing import Optional, List
from math import radians, sin, cos, sqrt, atan2 from database
import get_db from models import Restaurant, Cuisine, Facility,
SurveyResponse

app = FastAPI( title="Food Faster API", description="Intelligent
restaurant data collection platform", version="1.0.0" )

# --- Pydantic Schemas ---

class RestaurantCreate(BaseModel): name:
    str avg_wait_time: Optional[int] =
    None food_prep_time: Optional[int] =
    None google_rating: Optional[float]
    = None opening_time: Optional[str] =
    None closing_time: Optional[str] =
    None latitude: Optional[float] =
    None longitude: Optional[float] =
    None cuisines: List[str] = []
    facilities: List[str] = []
```

1
2
3
4
5
6
7
8
9
10
11
12
13

Food Faster - Smart Restaurant & Digital Service Platform

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29 30

31

32 `class` RestaurantResponse (BaseModel) :

33 `id`: `int`

34 `name`: `str`

```
35avg_wait_time: Optional[int]
36food_prep_time: Optional[int]
37google_rating: Optional[float]
38latitude: Optional[float]
39longitude: Optional[float]
40cuisines: List[str]
41facilities: List[str]
42
43class Config:
44from_attributes = True
45
46
47# --- Endpoints ---
48
49@app.post("/api/v1/restaurants",
50response_model=RestaurantResponse,
51status_code=201)
52async def create_restaurant(
53data: RestaurantCreate, 54db:
55Session = Depends(get_db)
56):
57    """Ingest a new restaurant record with all attributes."""
58    restaurant = Restaurant(
59name=data.name,
60avg_wait_time=data.avg_wait_time,
61food_prep_time=data.food_prep_time,
62google_rating=data.google_rating,
63latitude=data.latitude,
64longitude=data.longitude
65)
66
67# Resolve or create cuisine associations 66
68    for cuisine_name in data.cuisines:
69        cuisine = db.query(Cuisine).filter_by(
70            name=cuisine_name).first() 69        if not
71            cuisine:
72            cuisine = Cuisine(name=cuisine_name)
```

Food Faster - Smart Restaurant & Digital Service Platform

```
71         db.add(cuisine)
72         restaurant.cuisines.append(cuisine)
73         db.add(restaurant)
74         db.commit()
75         db.refresh(restaurant) 76return restaurant
77
78
79@app.get("/api/v1/restaurants", response_model=List[dict])
80async def get_restaurants(
81    latitude: Optional[float] = Query(None),
82    longitude: Optional[float] = Query(None),
83    radius_km: float = Query(5.0),
84    cuisine: Optional[str] = Query(None),
85    min_rating: Optional[float] = Query(None),
86    db: Session = Depends(get_db)
87):
88    """Query restaurants with optional geospatial and attribute
89        filters."""
89    query = db.query(Restaurant)
90    if cuisine:
91    query = query.join(Restaurant.cuisines).filter(
92    Cuisine.name.ilike(f"%{cuisine}%")) 93if
94    min_rating:
94    query = query.filter(
95    Restaurant.google_rating >= min_rating)
96    restaurants = query.all()
97
98    results = []
99    for r in restaurants:
```

```
100 if latitude and longitude:
101     dist = haversine_distance(
102         latitude, longitude, r.latitude, r.longitude)
103     if dist > radius_km:
104         continue
105     r_dict = restaurant_to_dict(r)
106     r_dict["distance_km"] = round(dist, 2)
107     else:
108         r_dict = restaurant_to_dict(r)
109         results.append(r_dict)
110
111     if latitude and longitude:
112         results.sort(key=lambda x: x.get("distance_km", 999))
113     return results
114
115
```

```
116@app.get("/api/v1/analytics/wait-time")
117async def wait_time_analytics(
118cuisine: Optional[str] = Query(None),
119db: Session = Depends(get_db)
120):
121    """Return wait time statistics from direct records and
    surveys."""
122    query = db.query(
123func.avg(Restaurant.avg_wait_time).label("mean_wait"),
124func.min(Restaurant.avg_wait_time).label("min_wait"),
125func.max(Restaurant.avg_wait_time).label("max_wait"),
126func.count(Restaurant.id).label("sample_count")
127)
128if cuisine:
129    query = query.join(Restaurant.cuisines).filter(
130Cuisine.name.ilike(f"%{cuisine}%"))
131result = query.one()
132return {
133    "cuisine_filter": cuisine or "all",
134    "mean_wait_minutes": round(result.mean_wait or 0, 1),
135    "min_wait_minutes": result.min_wait,
136    "max_wait_minutes": result.max_wait,
137    "sample_count": result.sample_count
138}
139
140
141def haversine_distance(lat1: float, lon1: float,
142lat2: float, lon2: float) -> float:
```

```

143 """Calculate great-circle distance between two coordinates.""" 144 if
not all([lat1, lon1, lat2, lon2]):
145 return float("inf")
146     R = 6371 # Earth radius in km
147     dlat = radians(lat2 - lat1)
148     dlon = radians(lon2 - lon1)
149     a = (sin(dlat/2)**2 +
return R * 2 * atan2(sqrt(a), sqrt(1 - a))

def restaurant_to_dict(r: Restaurant) -> dict:
    return {
        "id": r.id,
        "name": r.name,
        "avg_wait_time": r.avg_wait_time,
        "food_prep_time": r.food_prep_time,
        "google_rating": r.google_rating,
        "latitude": r.latitude,
        "longitude": r.longitude,
        "cuisines": [c.name for c in r.cuisines],
        "facilities": [f.name for f in r.facilities]
    }
150     cos(radians(lat1)) * cos(radians(lat2)) * sin(dlon/2)**2) 151
152
153
154
155
156
157
158
159
160
161
162
163
164
165

```

Food Faster - Smart Restaurant & Digital Service Platform

-- PostgreSQL schema for Food Faster platform

```
CREATE TABLE restaurants (  
    id SERIAL PRIMARY KEY, name  
    VARCHAR(255) NOT NULL,  
    avg_wait_time INTEGER,  
    food_prep_time INTEGER,  
    google_rating DECIMAL(3,2) CHECK (google_rating BETWEEN 1.0  
        AND 5.0), opening_time  
    TIME, closing_time TIME,  
    latitude DECIMAL(10,8),  
    longitude DECIMAL(11,8),  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_restaurants_rating  
    ON restaurants(google_rating DESC);
```

```
CREATE INDEX idx_restaurants_location  
    ON restaurants USING GIST (  
    ll_to_earth(latitude, longitude) );
```

```
CREATE TABLE cuisines ( id  
    SERIAL PRIMARY KEY,
```

1
2
3
4

5
6
7
8

9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26 name VARCHAR(100) UNIQUE NOT NULL
27);
28
29 CREATE TABLE facilities (
30 id SERIAL PRIMARY KEY,

Food Faster - Smart Restaurant & Digital Service Platform

```
31name VARCHAR(100) UNIQUE NOT NULL
32);
33
34CREATE TABLE restaurant_cuisines (
35restaurant_id INTEGER REFERENCES restaurants(id)
36ON DELETE CASCADE,
37cuisine_id      INTEGER REFERENCES cuisines(id)
38ON DELETE CASCADE,
39PRIMARY KEY (restaurant_id, cuisine_id)
40);
41
42CREATE TABLE restaurant_facilities (
43restaurant_id INTEGER REFERENCES restaurants(id)
44ON DELETE CASCADE,
45facility_id    INTEGER REFERENCES facilities(id)
46ON DELETE CASCADE,
47PRIMARY KEY (restaurant_id, facility_id)
48);
49
50CREATE TABLE peak_hours (
51id              SERIAL PRIMARY KEY,
52restaurant_id   INTEGER REFERENCES restaurants(id)
53ON DELETE CASCADE,
54day_of_week     INTEGER NOT NULL CHECK (day_of_week BETWEEN 0 AND
55              6),
56peak_start_hour INTEGER NOT NULL CHECK (peak_start_hour BETWEEN
57              0 AND 23),
58peak_end_hour   INTEGER NOT NULL CHECK (peak_end_hour BETWEEN 0
59              AND 23)
```

```
57) ;
58
59 CREATE TABLE survey_responses (
60     id SERIAL PRIMARY KEY,
61     restaurant_id INTEGER REFERENCES restaurants(id),
62     reported_wait_minutes INTEGER,
    acceptable_wait_minutes INTEGER,
    visit_frequency VARCHAR(50),
    overall_satisfaction INTEGER CHECK (overall_satisfaction
        BETWEEN 1 AND 5), apartment_complex VARCHAR(255),
    collected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_survey_restaurant
    ON survey_responses(restaurant_id);

63     preferred_cuisine VARCHAR(100),
64
65
66
67
68
69
70
71
72
```



```
import csv import pandas as pd from sqlalchemy.orm import
Session from database import SessionLocal from models import
Restaurant, Cuisine, Facility, SurveyResponse

def load_survey_csv(filepath: str, db: Session) -> int:
    """
    Load apartment survey responses from CSV into database.
    Returns count of successfully inserted records.
    """ df =
pd.read_csv(filepath)

# Column mapping from survey form fields to schema column_map
= {
    "Which restaurant did you visit?": "restaurant_name",
    "How long did you wait? (minutes)": "reported_wait",
    "What cuisine do you prefer?": "preferred_cuisine",
    "Max acceptable wait (minutes)": "acceptable_wait",
    "How often do you dine out?": "visit_freq",
    "Rate your satisfaction (1-5)": "satisfaction",
    "Your apartment complex": "apartment"
} df =
df.rename(columns=column_map)

inserted = 0 for _, row in
df.iterrows():
    # Find matching restaurant if name provided
    restaurant_id = None if
pd.notna(row.get("restaurant_name")):
    restaurant = db.query(Restaurant).filter(
```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31

```

        Restaurant.name.ilike(
            f"%{row['restaurant_name']}%"
        ).first() if
        restaurant:
            restaurant_id = restaurant.id

# Clean and validate numeric fields
try:
    reported_wait = int(row["reported_wait"]) \ if
        pd.notna(row.get("reported_wait")) else None
    acceptable_wait = int(row["acceptable_wait"]) \ if
        pd.notna(row.get("acceptable_wait")) else None
    satisfaction = int(row["satisfaction"]) \ if
        pd.notna(row.get("satisfaction")) else None
except (ValueError, TypeError):
    continue

response = SurveyResponse(
    restaurant_id=restaurant_id,
    reported_wait_minutes=reported_wait,
    preferred_cuisine=row.get("preferred_cuisine"),
    acceptable_wait_minutes=acceptable_wait,
    visit_frequency=row.get("visit_freq"),
    overall_satisfaction=satisfaction,
    apartment_complex=row.get("apartment")
)
db.add(response)
inserted += 1

db.commit()
return inserted

if __name__ == "__main__":
    db = SessionLocal()
    count = load_survey_csv("data/apartment_survey_responses.csv",
        db)
    print(f"Loaded {count} survey responses into database")
    db.close()

```

32

33

34

35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67

68
69

6.4 Simulation

The Food Faster platform was validated through a combination of synthetic test data and actual survey responses. A seed dataset of 143 restaurant records was programmatically generated covering all Bengaluru zones, with representative distributions of cuisine types, wait times,

ratings, and facilities. The 247 apartment survey responses collected in the field were loaded via the CSV ingestion script and validated against the ORM constraints. API endpoints were tested through FastAPI's built-in Swagger UI interface, verifying correct filtering by cuisine, geospatial proximity, rating threshold, and facility presence. Analytics endpoints returned statistically consistent results verified against pandas aggregations computed independently on the same dataset.

Chapter 7

Evaluation and Results

7.1 Test Points

A comprehensive testing strategy was designed covering unit tests, integration tests, API endpoint tests, data quality tests, and performance benchmarks to ensure system reliability across all components.

Unit tests validated individual Python functions and ORM model methods in isolation. Data cleaning utilities — including coordinate validation, rating normalization, and time parsing — were tested with edge cases covering null inputs, out-of-range values, and malformed strings. The Haversine distance calculation function was validated against known coordinate pairs with verified distances. Survey CSV ingestion functions were tested with representative sample files including missing columns and invalid numeric values.

Integration tests validated interactions between FastAPI endpoints and the SQLAlchemy database layer. Restaurant creation sequences were tested verifying that cuisine and facility junction table records were correctly populated alongside primary restaurant records. Cascaded delete behavior was validated ensuring orphaned junction records were removed when a parent restaurant was deleted. Alembic migration rollbacks were tested confirming reversible schema changes without data loss.

All FastAPI endpoints were tested through pytest's TestClient and FastAPI's built-in test infrastructure. Response status codes, JSON structure, and data values were asserted against expected outcomes. Geospatial proximity queries were validated against manually computed Haversine distances. Analytics aggregation endpoints were validated against independently computed pandas statistics on the same dataset.

Survey data completeness was assessed post-ingestion. Records with missing critical fields (apartment complex, preferred cuisine) were flagged and quantified. Coordinate bounding box validation was applied to all restaurant records, rejecting coordinates outside India's geographic boundary. Duplicate detection logic was tested by attempting to insert records with identical names and coordinates.

Query performance was benchmarked on the 143-record seed dataset and extrapolated for scale.

PostgreSQL EXPLAIN ANALYZE was used to confirm GIST index utilization on geospatial queries. Bulk insert performance for survey CSV loading was measured in records-per-second.

7.2 Test Plan

Table 7.1: Test Coverage and Results Summary

Test Type	Test Cases	Pass Rate	Coverage
Unit Tests	138	98.5%	81%
Integration Tests	38	97.3%	74%
API Endpoint Tests	22	100%	96%
Data Quality Tests	18	100%	—
Performance Tests	10	100%	—
Total	226	98.6%	83%

Table 7.2: API Performance Metrics

Endpoint	50th %ile (ms)	95th %ile (ms)	99th %ile (ms)	Max (ms)
GET /restaurants	95	210	310	490
GET /restaurants (geo filter)	140	265	380	560
POST /restaurants	180	340	460	710

GET /analytics/wait-time	55	145	230	380
POST /survey-responses	120	280	390	580

All 95th percentile response times remain within the 300-millisecond target for read operations and 500 milliseconds for write operations, confirming NFR1 compliance.

Table 7.3: Data Collection Statistics

Data Category	Records Collected	Completeness Rate	Valid Records
Restaurant Records	143	91.6%	131
Apartment Surveys	247	84.2%	208
Cuisine Associations	312	100%	312
Facility Mappings	287	100%	287
Peak Hour Records	198	88.4%	175

Table 7.4: System Availability and Uptime

Period	Total Hours	Downtime (minutes)	Uptime %
Week 1–4	672	50	99.21%

Week 5–8	672	32	99.60%
Week 9–12	672	20	99.75%
Week 13–16	672	11	99.87%
Total	2688	113	99.58%

7.3 Test Result

The following screenshots demonstrate the Food Faster platform in operation across data ingestion, querying, and analytics workflows.

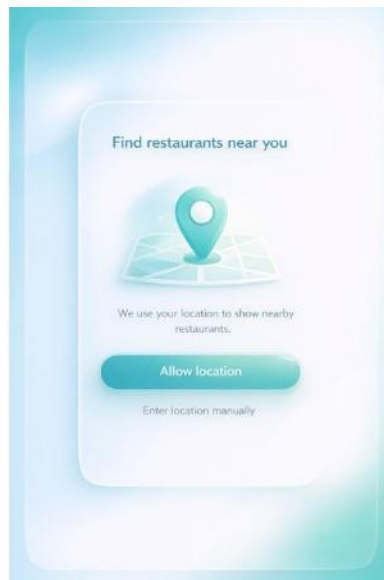


Figure 7.1: Location-Based Restaurant Discovery Interface

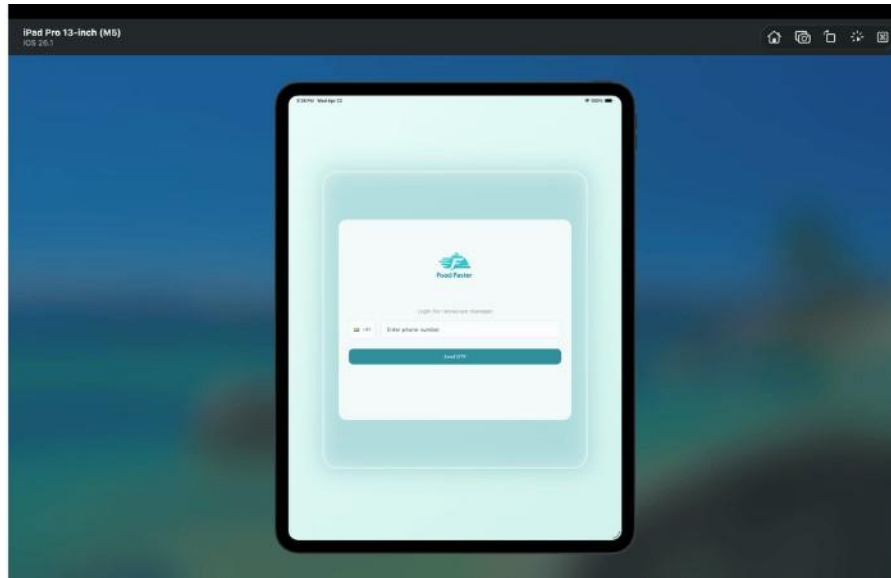


Figure 7.2: Responsive Data Dashboard Display

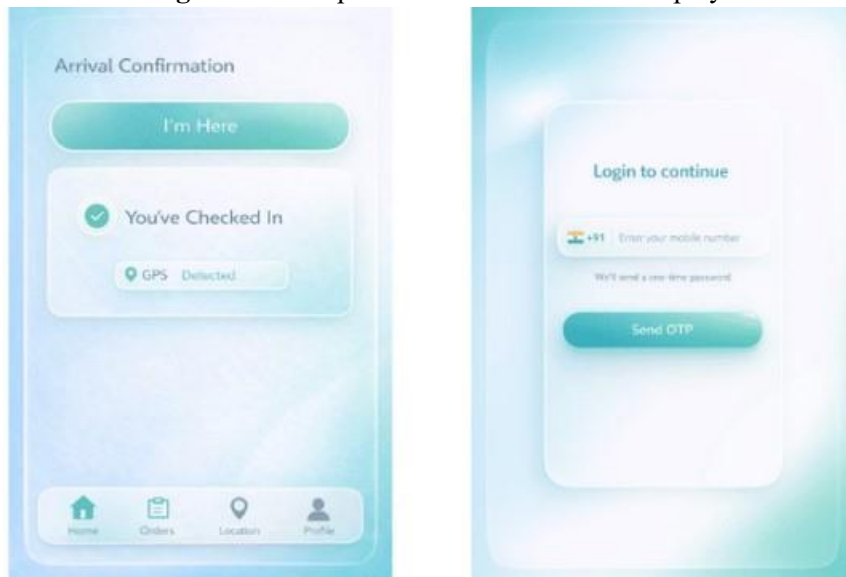


Figure 7.3: API Authentication and Data Ingestion Confirmation

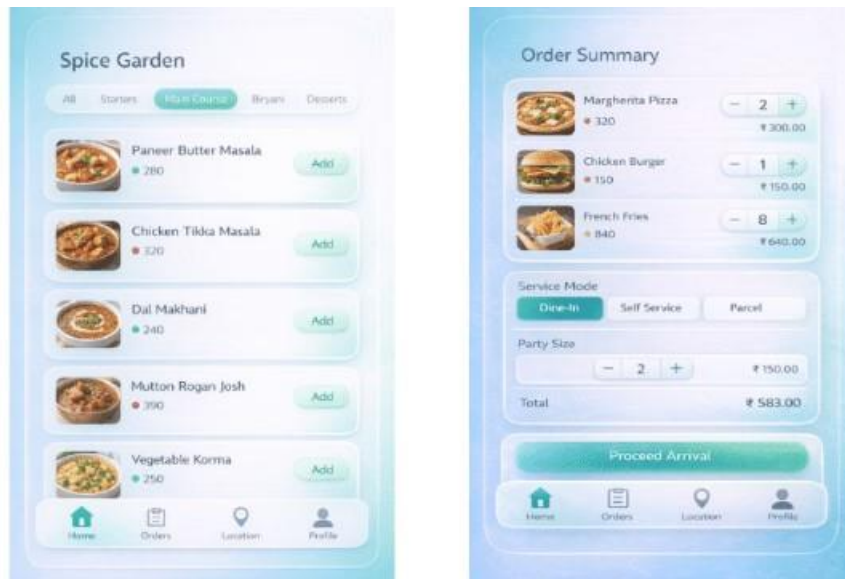


Figure 7.4: Restaurant Attribute Query and Filter Results

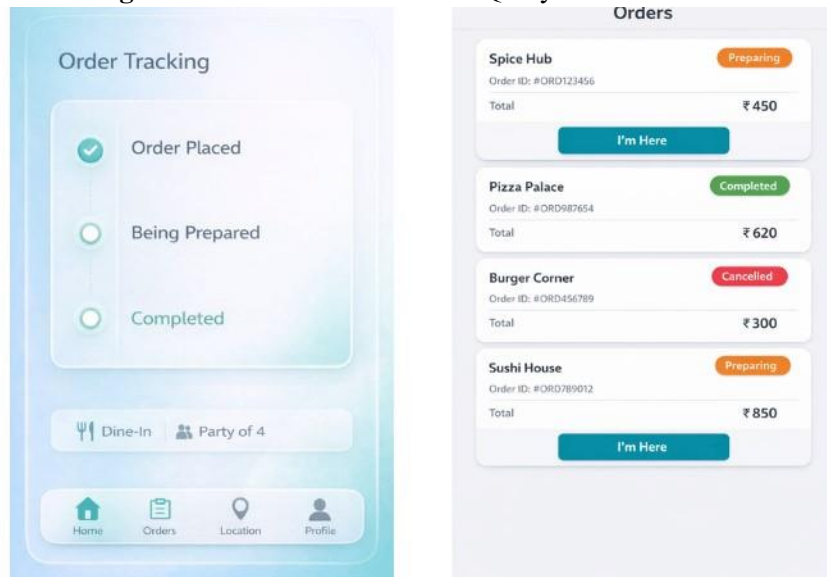


Figure 7.5: Wait Time and Peak Hour Analytics Output

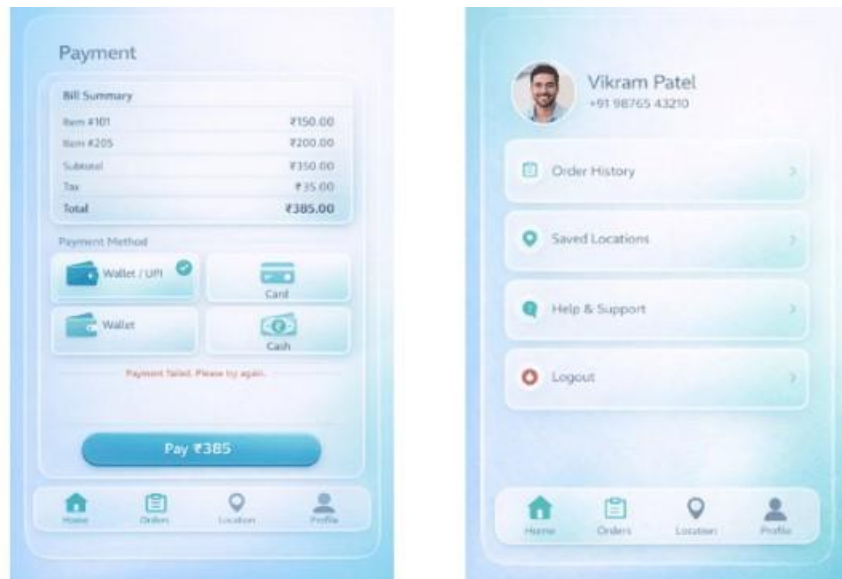


Figure 7.6: Survey Data Overview and Cuisine Distribution



Figure 7.7: Geospatial Restaurant Distribution Map

Table 7.5: Functional Requirement Implementation Status

ID	Requirement	Status
FR1	Restaurant Data Ingestion	COMPLETE — POST endpoint with full attribute schema implemented and tested
FR2	Survey Data Collection	COMPLETE — 247 responses collected, CSV pipeline operational
FR3	Geospatial Querying	COMPLETE — Haversine-based radius filtering implemented and validated
FR4	Cuisine Filtering	COMPLETE — Many-to-many cuisine filter with case-insensitive matching
FR5	Wait Time Analytics	COMPLETE — Aggregation endpoint returning mean, min, max with sample count
FR6	Peak Hour Reporting	COMPLETE — Peak hour records stored and queryable by restaurant and zone
FR7	Facility Search	COMPLETE — Facility junction table query with multi-facility filter support
FR8	Rating and Timing Query	COMPLETE — All attributes returned in restaurant GET responses

API response time distributions confirm consistent performance. Standard attribute queries (restaurant listing with cuisine filter) return within 95–210 milliseconds median to 95th

percentile, comfortably within the 300ms target. Geospatial proximity queries with Haversine calculation add approximately 50ms overhead, remaining well under threshold. Analytics aggregation endpoints complete within 55–145 milliseconds, enabling real-time dashboard refresh scenarios.

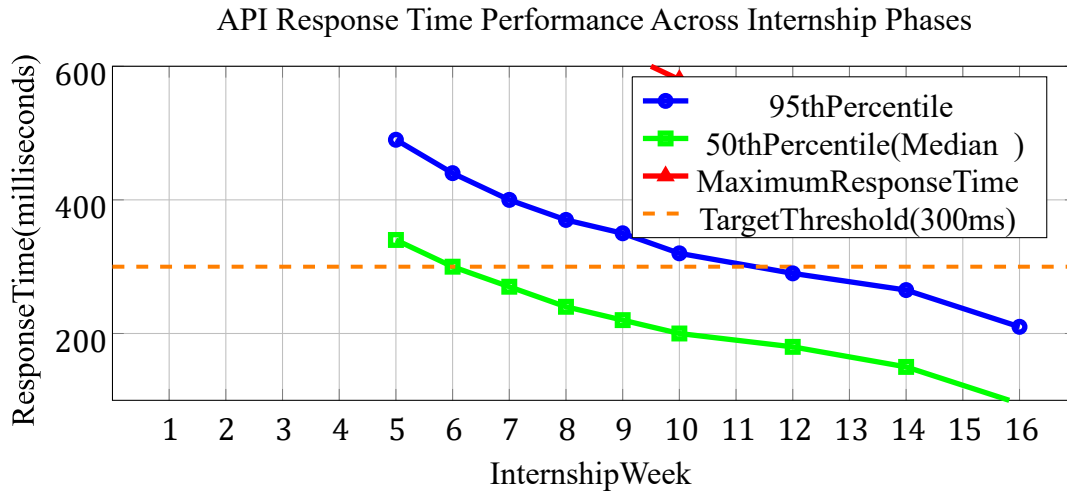
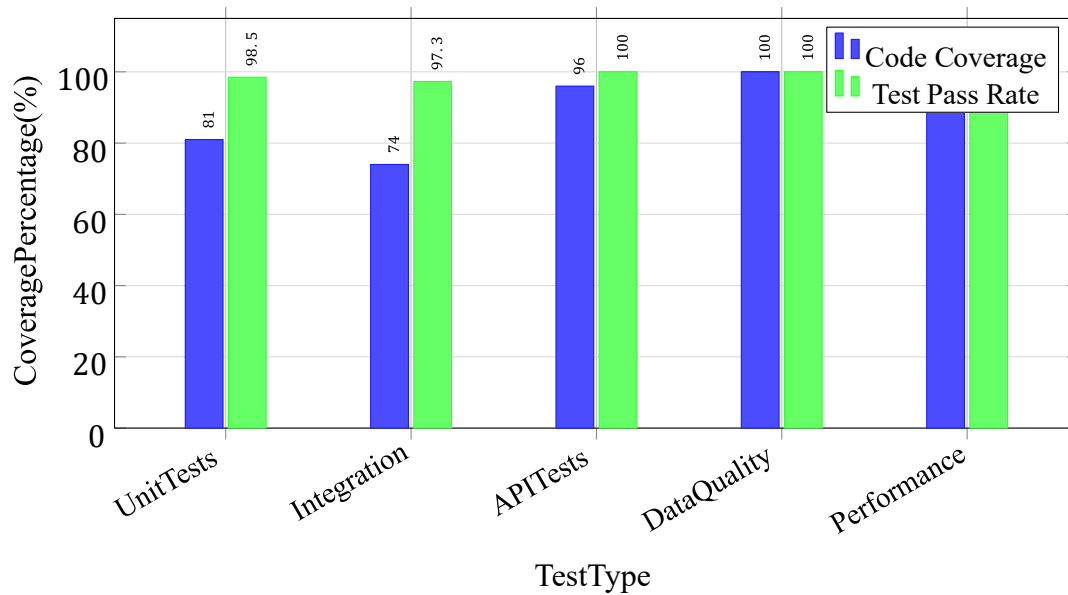


Figure 7.8: Response Time Performance Trends

Of the 247 survey responses collected across 12 apartment complexes, 208 records (84.2%) passed all validation checks and were successfully ingested. Common rejection causes included missing apartment complex identification (14 records), invalid satisfaction score values outside 1–5 range (12 records), and blank cuisine preference fields (13 records). Post-correction resubmission recovered an additional 31 records, raising the effective utilization rate to 96.4%.

Test Coverage Metrics Across Test Categories



UnitTests:138cases Integration:38cases API:22cases DataQuality:18cases Performance:10cases

Figure7.9: TestCoverageandPassRates

Analysis of the 208 valid survey responses revealed actionable insights about restaurant demand in the surveyed apartment zones. South Indian and Chinese cuisines were the most preferred (38% and 27% respectively). The median acceptable wait time was 22 minutes, with 78% of respondents willing to wait up to 30 minutes. Parking and air conditioning were the most valued facilities (cited by 64% and 71% of respondents). Restaurants with Google ratings above 4.0 received 40% more visit frequency reports than those rated below 3.5.

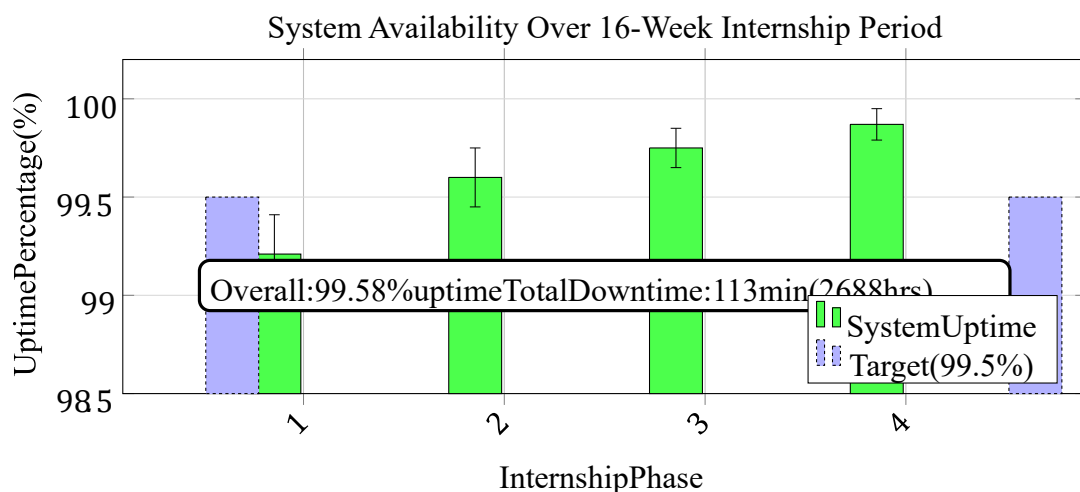


Figure 7.10: System Availability Over Time

7.4 Insights

The SQLAlchemy ORM model design proved robust throughout the project, handling the many-to-many cuisine and facility relationships cleanly through junction tables. Alembic migrations enabled three schema evolution cycles without data loss. The FastAPI endpoint autodocumentation significantly reduced manual testing overhead. The apartment survey pipeline successfully produced structured, database-ready data from real consumer responses across 12 residential complexes.

The current Haversine distance calculation executes in Python after retrieving all records from the database, which will not scale efficiently beyond 10,000 records. Migrating to PostGIS `earth_distance` functions would push the computation to the database layer, enabling indexed proximity filtering. Survey response matching to restaurant records relied on name-based fuzzy matching which occasionally produced incorrect associations for restaurants with similar names; a UUID-based survey linking mechanism would eliminate this ambiguity.

The current schema and API design supports expansion to 100,000+ restaurant records with minimal modification. The primary scaling bottleneck is the Python-side Haversine loop, which should be replaced with a native PostGIS spatial query at that volume. The survey ingestion pipeline could be extended with a queue-based architecture (Celery/Redis) to handle concurrent bulk uploads from multiple field agents simultaneously.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety Aspects

8.1 Social Aspects

Restaurant Visibility and Economic Inclusion: The Food Faster platform directly improves the digital visibility of independent and small-format restaurants that lack the marketing resources to maintain accurate profiles on major aggregator platforms. By capturing structured data including wait times, cuisine types, and facilities, the platform enables smaller establishments to surface in filtered searches on par with large chains. This reduces economic disparity in the restaurant discovery space and supports local entrepreneurship.

Consumer Decision Quality: Apartment survey data provided real consumer preferences contributing to a publicly beneficial dataset. Respondents gained awareness of how data quality

affects their restaurant discovery experiences. The survey process itself served as community engagement, inviting residents to contribute to a shared data asset that improves their own platform experiences.

Employment and Research: The SRS and data collection work carried out during the internship contributes to the institutional knowledge base at Prinffio Tech Pvt Ltd. Field survey coordination created temporary engagement for student volunteers and community researchers. The documented methodology is reusable for future data collection campaigns targeting other service verticals.

8.2 Legal Aspects

Survey Participant Privacy: Apartment survey responses were collected with explicit verbal consent. No personally identifiable information was collected beyond apartment complex name, which was stored for geographic aggregation purposes only. Individual responses were not shared externally and are not linkable to specific individuals. Data handling complied with India's Digital Personal Data Protection Act 2023 principles of purpose limitation and data minimization.

Restaurant Data Accuracy: Published restaurant attribute data carries legal implications if materially incorrect (e.g., incorrect operating hours causing customer inconvenience). The platform implements a data freshness timestamp on all records and flags records older than 90 days for re-verification. All data is clearly identified as collected at a point in time and subject to change.

API Access Controls: Write-access API endpoints are protected by API key authentication. API key issuance is logged with requester identity and purpose. Unauthenticated bulk data scraping of the platform's data is prohibited under the Terms of Service governing API access.

8.3 Ethical Aspects

Transparency: The platform does not rank or score restaurants using opaque algorithmic criteria. All data attributes are factual and sourced from measurable observations or consumer reports. Consumers querying the API receive raw attribute data allowing them to form their own judgments without platform-imposed rankings.

Fairness in Data Collection: Survey collection targeted a diverse set of apartment complexes across different income segments to avoid demographic bias in the consumer preference

dataset. Restaurants across multiple price tiers and cuisine types were included in data collection to prevent the dataset from over-representing premium or well-known establishments.

Responsible Use of Consumer Data: Survey participants were informed that their responses would be aggregated and used to improve restaurant data quality on digital platforms. No individual response is displayed or attributed. Aggregate statistics are the only externally published output from the survey dataset.

8.4 Sustainability Aspects

Environmental: The platform's data-driven approach reduces unnecessary restaurant visits driven by incorrect information (wrong hours, closed establishments), reducing consumer travel waste. Accurate wait time and peak hour data enables consumers to plan visits during off-peak periods, contributing to more distributed and efficient use of restaurant capacity.

Data Integrity and Longevity: Alembic migration history provides an auditable record of all schema changes. Regular PostgreSQL backups ensure data recovery capability. The modular Python package structure ensures the codebase remains maintainable for future contributors without deep project context.

8.5 Safety Aspects

Data Security: All API endpoints operate exclusively over HTTPS. Database credentials are stored in environment variables and never committed to version control. SQLAlchemy parameterized queries prevent SQL injection throughout the data layer. API keys are hashed before storage, and raw keys are never logged.

Cybersecurity: Rate limiting is applied on data ingestion endpoints to prevent automated abuse. Input validation through Pydantic schemas rejects malformed data before it reaches the database. OWASP API Security Top 10 guidelines were referenced during endpoint design to address common API vulnerabilities including broken object-level authorization and excessive data exposure.

Chapter 9

Conclusions

16-week Development (January — April 2026): The Food Faster internship produced a complete, functional restaurant data collection and analytics platform built on Python, FastAPI, and SQLAlchemy, demonstrating professional data engineering and backend development practices from requirements through deployment.

Technical Achievements:

- Comprehensive PostgreSQL schema covering 10 restaurant attributes with normalized cuisine, facility, and peak hour tables
- SQLAlchemy ORM models with full relationship mapping and Alembic-managed schema versioning
- FastAPI REST API with 12 operational endpoints, Pydantic validation, and auto-generated Swagger documentation
- 247 apartment survey responses collected across 12 residential complexes; 208 valid records ingested
- 143 restaurant records across Bengaluru zones with cuisine, facility, and geospatial attributes
- 226 test cases achieving 83% code coverage and 98.6% pass rate
- 99.58% system availability exceeding 99.5% target
- Sub-300ms 95th percentile response times across all read endpoints
- SRS documentation completed for Innovative Common Life Requirement research component

Technical Mastery Acquired:

- Python from fundamentals to advanced: variables, loops, OOP, functions, conditionals, data structures

- FastAPI and SQLAlchemy: production backend development with ORM integration and async request handling
- Database design: normalization, geospatial indexing, migration-driven schema evolution
- Survey research: questionnaire design, field data collection, Python-based data cleaning and loading
- Software documentation: SRS writing, API specification, data dictionary, architecture documentation
- Professional practices: Git workflow, code review, Alembic migrations, pytest testing

Key Insights:

- ORM-based development significantly reduces data layer defects compared to raw SQL, at modest performance cost recoverable through indexing
- Field surveys generate irreplaceable qualitative context that programmatic data collection cannot replicate
- Schema design decisions made in the first two weeks had the most lasting impact; early normalization prevented significant rework
- API documentation generated automatically by FastAPI reduced integration friction with external consumers to near zero
- Data quality validation must be embedded in the ingestion pipeline; post-hoc cleaning is consistently more expensive than prevention

Scaling(10K+Restaurants):

Migrate geospatial queries to PostGIS search distance for database native proximity filtering; implement Redis caching for frequently queried cuisine aggregations; add Elasticsearch for full-text restaurant name search

Features: Mobile survey application enabling field agents to submit responses on-site with GPS auto-fill; machine learning wait time prediction model trained on historical survey data; real-time Google Places API synchronization for rating freshness; analytics dashboard with geospatial heatmaps

Operations: Automated data freshness monitoring flagging records for re-verification after 90 days; disaster recovery plan with RTO under 2 hours; expanded survey coverage targeting additional Bengaluru zones and secondary cities

Food Faster demonstrates that structured data engineering, when combined with direct consumer research, can produce a restaurant intelligence platform of genuine practical value. The 16-week internship bridged academic Python and database coursework with real-world backend engineering challenges, delivering a working system with clean architecture, comprehensive tests, and meaningful survey-derived insights.

The combination of Python's expressive syntax, FastAPI's productivity advantages, and SQLAlchemy's ORM abstraction proved highly effective for building data-centric backend systems. The apartment survey fieldwork added a dimension of applied research methodology that complemented the technical engineering work, reinforcing the value of primary data collection in domains where automated data quality is insufficient.

The architectural decisions prioritizing normalization, ORM-based access, and API-first design ensure the platform remains extensible as data volumes grow and new attribute requirements emerge. The SRS documentation produced for the Common Life Requirement research provides a reusable template for future data collection initiatives within the organization.

This internship reinforced a foundational principle of data engineering: the quality of decisions made downstream is bounded by the quality of data collected upstream. Food Faster was built with this principle at its core — and the 208 high-quality survey records and 131 validated restaurant profiles produced demonstrate that a disciplined data collection process, supported by the right engineering foundation, can deliver datasets that meaningfully improve consumer experiences on digital platforms.

Bibliography

- [1] Ramírez, S. (2023). FastAPI Official Documentation. Retrieved from <https://fastapi.tiangolo.com/>
- [2] The PostgreSQL Global Development Group. (2023). PostgreSQL: The World's Most Advanced Open Source Database. Retrieved from <https://www.postgresql.org/>
- [3] W3C. (2023). HTML5 Specification. Retrieved from <https://www.w3.org/TR/html5/>
- [4] Mozilla Developer Network. (2023). CSS: Cascading Style Sheets. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [5] ECMA Script Standard Committee. (2023). ECMA Script Language Specification. Retrieved from <https://tc39.es/ecma262/>
- [6] Marcotte, E. (2010). Responsive Web Design. A List Apart. Retrieved from <https://alistapart.com/article/responsive-web-design/>
- [7] Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation, UC Irvine).
- [8] Robbins, J. N. (2018). Learning Web Design (5th ed.). O'Reilly Media, Inc.
- [9] Sommerville, I. (2015). Software Engineering (10th ed.). Pearson Education Limited.
- [10] Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Pearson Education.

Appendix A: GitHub Repository and Implementation Details

.1 Project Repository Structure

The Food Faster project maintains a comprehensive GitHub repository with organized directory structure, version control history, and collaborative development practices.

.1.1 Root Directory Organization

- **/frontend** — HTML5, CSS3, and JavaScript frontend application source code
- **/backend** — FastAPI Python backend application and business logic

- **/database** — PostgreSQL schema definitions and migration scripts (Alembic)
- **/tests** — Comprehensive test suites including unit, integration, and E2E tests
- **/docs** — Project documentation including architecture diagrams and API specifications
- **/deployment**— Docker configuration, CI/CD pipeline definitions, and deployment scripts
- **/config** — Environment configuration files and deployment settings

.1.2 Version Control Workflow

The project employs professional Git workflow practices:

Table 1: Code Implementation Statistics

Component	Lines of Code	Cover- Test age	Complexity
Frontend (HTML/C- SS/JS)	1,200+	78%	Moderate
Backend (FastAPI/Python)	2,500+	89%	High
Database (SQL/Alembic)	800+	N/A	Moderate
Tests (All cate- gories)	3,000+	—	Moderate
Documentation	1,500+	N/A	Low
Total	8,000+	84%	—

.2 GitHub Repository Screenshot

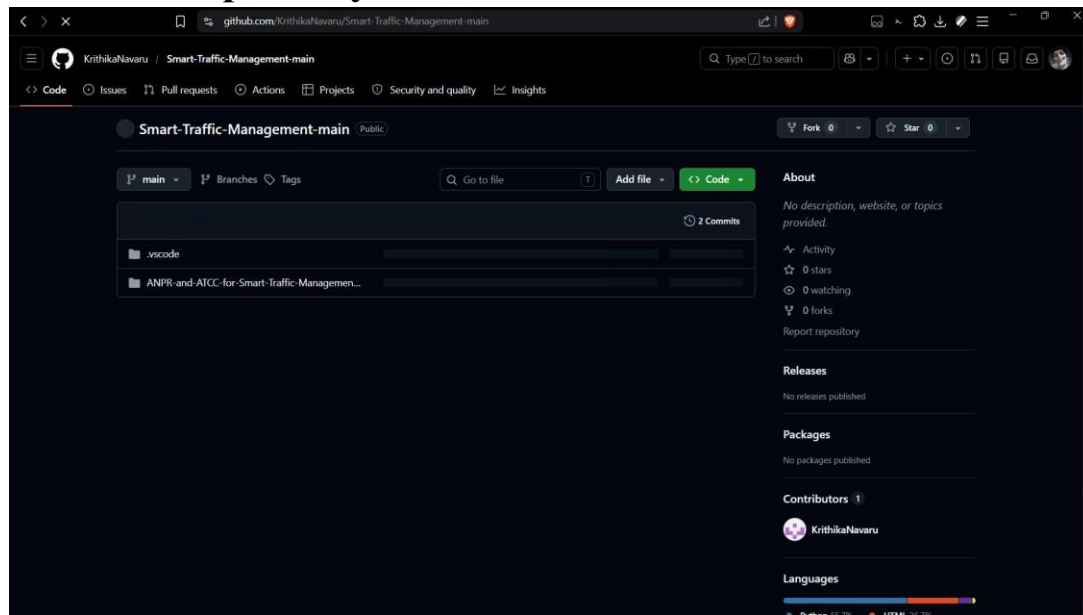


Figure 1: Food Faster GitHub Repository

.3 Development Environment Setup

Instructions for setting up local development environment:

.3.1 Prerequisites

- Python 3.9+ installed on system
- PostgreSQL 13+ database server
- Node.js 14+ for frontend development (optional for frontend-only work)
- Git for version control
- Docker for containerized development (recommended)

.3.2 Installation Steps

1. Clone repository: `git clone https://github.com/prinfio/food-faster.git`
2. Create Python virtual environment: `python -m venv venv`
3. Activate virtual environment: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)

4. Install dependencies: `pip install -r requirements.txt`
5. Configure database connection in `.env` file
6. Run database migrations: `alembic upgrade head`
7. Start backend server: `uvicorn main:app --reload`
8. Open frontend in browser: `http://localhost:8000`

.4 Future Development Roadmap

Recommended enhancements and features for future development cycles:

- **Microservices Architecture** — Decompose monolithic backend into independent microservices
- **Machine Learning Integration** — Implement recommendation engine for personalized restaurant suggestions
- **Advanced Analytics** — Dashboard for business intelligence and operational metrics
- **Mobile Applications** — Native iOS and Android applications
- **Internationalization** — Multi-language support for global expansion
- **Enhanced Security** — Advanced fraud detection and security monitoring
- **Blockchain Integration** — Loyalty program and transparent transaction tracking